

DEPLOYMENT PLAN

UMHackathon 2026

umhackathon@um.edu.my

1.0 Technical Stack & Scalability

1.1 Core Infrastructure & Hosting

Note: The current technological stack (including Vite, Railway, and Netlify) was strategically selected for the rapid development of this MVP during the 10-day hackathon cycle. A comprehensive roadmap for transitioning to enterprise-grade infrastructure (AWS/GCP) is outlined in Section 7.0.

This section describes the distributed and modular infrastructure that enables SCADA-i to operate seamlessly across its various cloud orchestration tiers.

Frontend Tier:

- **Technologies:** Built with React and TypeScript, utilizing Vite as the build tool and **Tailwind CSS** for styling.
- **Hosting:** Deployed on Netlify to provide a high-performance web interface for both the University User App and the Facility Manager Dashboard.

Backend & Orchestration Tier:

- **Environment:** Powered by Node.js with TypeScript and FastAPI.
- **Hosting:** Backend services and specialized agents are deployed on Railway.
- **Multi-Agent Logic:** Orchestrated via the LangGraph framework, potentially utilizing Z.AI GLM (currently gpt-4o for the Orchestrator and deepseek-chat for workers) as the reasoning service layer.

Data Persistence Tier:

- **Database:** A centralized PostgreSQL relational database manages state for room availability, HVAC setpoints, and booking records.
- **Cloud Provider:** Hosted on Supabase (AWS ap-south-1 region).
- **Policy Storage:** Uses Supabase Storage buckets for document management (e.g., ESG policies).

1.2 IoT Messaging & Communication

This section outlines how SCADA-i achieves real-time, autonomous hardware control.

- **Communication Protocol:** Utilizes the MQTT protocol for reliable, event-driven communication between AI agents and IoT devices.

- **IoT Broker:** A dedicated messaging tier (MQTT Broker) facilitates the transmission of real-time sensor data (occupancy, temperature) and the execution of HVAC commands.
- **Closed-Loop Execution:**
 1. **Ingestion:** Real-time sensor telemetry is transmitted via MQTT and processed by the backend services.
 2. **Optimization:** The HVAC Agent evaluates these conditions against ESG policies and uses a LightGBM model optimizer to generate refined commands.
 3. **Actuation:** Finalized commands are published back over MQTT to the hardware controllers (e.g., AC units).
- **Resilience & Safety:** To handle potential communication failures, the system includes **timeout constraints** (5-8 seconds) and manual override protections that temporarily pause automated optimization.

1.3 Database Architecture & State Management

This section describes the persistent data layer that enables the multi-agent system to maintain context and coordination.

- **Relational Data Storage:** SCADA-i utilizes a normalized PostgreSQL database hosted on Supabase (AWS ap-south-1) to ensure data integrity across building operations.
- **Schema Composition:** The schema is designed to support multiple domain-specific tables, including:
 - **Occupancy & Environmental State:** Real-time logging of room temperature, humidity, and active occupancy feeds for HVAC optimization.
 - **Booking Lifecycle:** Storage of room descriptions, equipment inventory, and department-specific booking records.
 - **ESG Compliance:** Tables for storing carbon offset calculations, emissions data, and weekly sustainability reports.
- **Real-time Data Synchronization:** The system leverages Supabase Realtime to push database updates (such as HVAC status changes or sensor anomalies) directly to the admin dashboard and mobile app via a websocket connection.
- **Persistence for Agents:** LangGraph state is persisted using an AsyncPostgresSaver, allowing the Orchestrator to maintain "memory" of complex multi-step workflows even after system reboots.

1.4 Horizontal & Vertical Scalability

This section outlines how the system is architected to handle growth from a single prototype to large-scale facility management.

- **Modular Agent Scaling:** The system uses a multi-agent microservices architecture. Specialized agents (HVAC, Booking, ESG) are decoupled and deployed individually, allowing for horizontal scaling where specific worker nodes can be expanded based on demand.
- **Serverless Elasticity:** By deploying the LangGraph orchestrator and domain workers on Supabase Edge Functions and Railway, the system automatically scales compute resources up or down based on the volume of incoming API requests.
- **Messaging Throughput:** The MQTT-based messaging tier is designed for high-concurrency IoT telemetry. It allows thousands of sensors to publish data simultaneously without bottlenecking the central AI reasoning engine.

2.0 CI/CD Pipeline & Automation

2.1 Version Control & Branching Strategy

This section defines how the codebase is managed to support collaborative development among the agentic AI, frontend, and backend teams.

- **Centralized Repository:** All source code is hosted on **GitHub**, acting as the single source of truth for the system's distributed microservices.
- **Branching Model:** The project follows a structured branching strategy to maintain stability:
 - **Main Branch:** Reserved for stable, production-ready code used during the final demo.
 - **Develop Branch:** Used for day-to-day integration of the HVAC, Booking, and ESG sub-agents.
- **Team Contribution:** Specific roles (e.g., Agentic AI Developer Lead, Frontend Developer) manage their respective modules via feature branches to prevent code conflicts.

2.2 Automated Build & Test Workflows

Automation is used to validate the complex multi-agent reasoning paths before they reach the execution layer.

- **CI Tooling: GitHub Actions** is used to drive the automated pipeline, running linting and build checks on every push or pull request.
- **Validation Stages:**
 - **Independent Agent Validation:** HVAC, Booking, and ESG capabilities are tested in isolation before being integrated into the LangGraph orchestrator.
 - **Scenario Validation:** Automated tests evaluate end-to-end flows across success, blocked, and fallback paths.
 - **Stability Checks:** Repeated prompt runs are executed to verify that the GLM produces consistent tool invocations and predictable routing behavior.
- **Guardrail Safety Checks:** The pipeline includes automated policy-bounded control logic to ensure that agents do not generate unsafe HVAC setpoints or bypass governance rules.

2.3 Deployment Automation

The transition from code to cloud is handled through continuous deployment to minimize manual intervention.

- **Automated Hosting Integration:** Once the build and test stages pass, the system automatically triggers deployments to the respective cloud tiers:
 - **Frontend:** Automatically deployed to Netlify.
 - **Backend & Agents:** Automatically deployed to Railway.
 - **Database & Edge Functions:** Synced with the Supabase infrastructure.
- **Environment Consistency:** Deployment automation ensures that the system architecture (distributed across Frontend, Orchestrator, Worker, Data, and Messaging tiers) remains synchronized across all environments.
- **Emergency Fallback Procedures:** Deployment configurations include operational fail-safe procedures, such as ensuring the system defaults to a "safe state" if external LLM APIs or MQTT communication fails during rollout.

3.0 Integration Security & Credential Management

3.1 Identity & Access Management (IAM)

- **Role-Based Access Control (RBAC):** The system enforces a strict role-aware governance model. Administrative users (Facility Managers) have access to global energy optimization dashboards and ESG reporting, while standard users (Students/Staff) are restricted to personal room booking and local comfort overrides.
- **Administrative Gatekeeping:** The **LangGraph** supervisor uses a specialized `build_supervisor_prompt()` that includes administrative gatekeeping rules. This prevents non-admin users from accessing sensitive reporting tools or policy-sensitive HVAC commands.
- **User Context Injection:** To prevent session hijacking or unauthorized data access, the Backend API secretly injects the verified `user_id` into every message before it is sent to the AI agents, ensuring the system remains grounded in the user's actual permissions.

3.2 Logic Guardrails & Operational Safety

- **Policy-Bounded Control:** The system implements safety guardrails to prevent unsafe hardware operations. For example, any HVAC temperature request that falls outside of pre-defined safety bounds is automatically clamped to a policy-compliant value by the domain agent before the command is sent via MQTT.
- **Governance Rollbacks:** In the event of a policy-retrieval failure, the **ESG Agent** is programmed to block compliance-sensitive actions and return a governance-safe response rather than risk a policy violation.
- **Manual Override Protection:** To ensure human control over AI decisions, the system supports manual overrides that temporarily pause automated optimization, preventing conflicting actions between the AI agent and the building technician.

3.3 Data & Communication Security

- **Secure Persistence:** Data integrity is managed through **Supabase** (PostgreSQL), utilizing its security features to isolate room states, booking details, and ESG policies. On any database write failure, the system performs a rollback to preserve the last known consistent state.

- **Real-time Messaging Security:** Communication between the AI layer and the physical hardware is coupled to **MQTT protocol contracts**. This event-driven architecture ensures that commands are structured and validated against specific device schemas before execution.
- **Token & Context Security:** To prevent token-based attacks or runaway costs, the orchestrator utilizes a local token counter (tiktoken) to strictly enforce a 4,000-token bound per invocation, preventing the system from processing excessively large or malicious inputs.

4.0 Production Architecture & Deployment Timeline

4.1 Technical Evaluation & Emergency Rollback

To ensure the system produces reliable and grounded outputs, **SCADA-i** follows an incremental validation approach.

- **Testing Methodology:**
 - Capabilities for HVAC, Booking, and ESG are validated independently before multi-agent integration.
 - Repeated prompt runs verify stable routing behavior and consistent tool invocation.
- **Fail-Safe Mechanisms:**
 - **Optimizer Timeout:** If the LightGBM optimizer fails to respond within 5–8 seconds, the system blocks execution to avoid applying incorrect HVAC changes.
 - **Manual Override:** Automated optimization is paused when a user manually adjusts settings to prevent conflicting actions.
 - **Safe-State Defaults:** If the AI layer or external services fail, the system defaults to a "no action" safe state with a clear user-facing explanation.

4.2 System Monitoring & Audit Trails

The system includes comprehensive logging to ensure traceability across the distributed architecture.

- **Execution Tracking:** All requests are tracked using a unique request_id to enable traceability across the agent, service, and execution layers.
- **Real-Time State Monitoring:** Room states, including temperature and occupancy, are continuously updated in the database for anomaly detection and decision-making.

- **Lifecycle Logging:**
 - **Booking Events:** Updates, cancellations, and conflict failures are logged to improve recoverability and audit trails.
 - **ESG Actions:** Policy lookup attempts and retrieval outcomes are logged to ensure explainability in compliance decisions.

4.3 External Dependencies & Risk Management

The functionality of **SCADA-i** relies on several critical external integrations.

- **AI Reasoning Layer:** Depends on the potentially Z.AI GLM API (currently OpenAI and DeepSeek); failure results in the inability to process natural language requests.
- **Messaging & Storage:**
 - **MQTT Broker:** Vital for real-time hardware communication; delays affect system responsiveness.
 - **Supabase/PostgreSQL:** Essential for state consistency; downtime affects the ability to store bookings or ESG logs.
- **Retrieval Stack:** ESG grounding depends on the embedding API and vector store; poor retrieval can lead to degraded compliance confidence.

4.4 Deployment Timeline

The project follows a 10-day rapid development and deployment cycle.

Phase	Activities
Day 1-2	Problem analysis and system architecture design.
Day 3-6	Core development of HVAC, Booking, and ESG agents, alongside the mobile app and dashboard.

Day 7-8	Integration of sub-agents with the Z.AI GLM reasoning engine.
Day 9-10	End-to-end testing, debugging, and final demo preparation.

5.0 Monitoring & Maintenance

This section outlines the strategic roadmap to transition **SCADA-i** from a hackathon prototype to an enterprise-grade smart building ecosystem.

5.1 Scalability & Performance Optimization

- **Multi-Building Support:** Extend the current architecture to support campus-wide or multi-city deployments by introducing a centralized global monitoring dashboard.
- **Distributed Caching:** Implement high-performance caching mechanisms (such as **Redis**) to store frequently accessed IoT telemetry and agent states, reducing repeated LLM API calls and lowering overall latency.
- **Advanced Load Balancing:** As the number of connected sensors increases, deploy a load-balanced MQTT broker cluster to ensure no single point of failure in the real-time messaging tier.

5.2 Reliability & Intelligence Improvements

- **Enhanced Fallback Redundancy:** Develop "Shadow Agents" or secondary LLM integrations (e.g., local Llama-3 models) that can take over basic HVAC and safety functions if the primary **Z.AI GLM API** becomes unavailable.
- **Predictive Analytics Integration:** Beyond real-time optimization, integrate deep learning models for long-term energy forecasting and predictive maintenance, allowing the system to identify potential hardware failures before they occur.
- **Natural Language Expansion:** In future versions, incorporate voice-to-text processing and speech recognition to allow building users to interact with **SCADA-i** via ambient room microphones or mobile voice commands.

5.3 UX & Compliance Evolution

- **Advanced ESG Dashboards:** Expand the current PDF-based reporting into a full interactive analytics suite, providing building owners with real-time carbon footprint visualizations and ESG "scorecards".
- **Autonomous Governance:** Refine the **ESG Agent's** RAG (Retrieval-Augmented Generation) pipeline to automatically ingest and update city-specific sustainability laws, ensuring the building remains compliant with changing regional regulations without manual intervention.

6.0 Enterprise Expansion & Commercial Deployment Plan

This section outlines the strategic roadmap to transition **SCADA-i** from a hackathon prototype into an enterprise-grade, Hardware-Enabled Software-as-a-Service (SaaS) platform for the B2B sector. The evolution focuses on centralized cloud orchestration, modular IoT hardware outfitting, and usage-based commercial scaling.

6.1 Centralized Enterprise Cloud & Multi-Tenancy

The system architecture will migrate to a multi-tenant model to support global university deployments from a unified hosting environment.

- **Centralized Orchestration:** Backend services and AI sub-agents will be containerized via **AWS EKS** or **GCP**, allowing the reasoning engine to serve multiple campuses with high availability.
- **Strict Data Isolation:** Data integrity is managed through a PostgreSQL database utilizing Row-Level Security (RLS). All environmental telemetry and booking records are cryptographically isolated via a unique `tenant_id`.
- **Global MQTT Infrastructure:** A load-balanced MQTT broker cluster will handle massive ingestion of real-time sensor data from concurrent institution-wide deployments.

6.2 Turnkey IoT Hardware Installation (Hardware-as-a-Service)

This section describes the end-to-end service for outfitting institutional facilities with autonomous IoT infrastructure.

- **Site Survey & Deployment:** Technical teams will perform site-specific audits to install environmental sensors (occupancy, humidity) and hardware actuators for real-time HVAC control.

- **Edge Gateway Processing:** To maintain safety during connectivity failures, each campus receives a localized Edge Gateway to provide fallback logic and batch sensor data for the central orchestrator.
- **Secure Tunneling:** Hardware communication is secured via encrypted VPN tunnels to prevent unauthorized tampering or local cybersecurity breaches.

6.3 White-Labeled & Custom UI Architecture

The system architecture supports Institution-specific customization while maintaining a shared backend orchestration layer.

- **Dynamic Theming Engine:** Utilizing the React and Tailwind CSS stack, the UI automatically updates brand colors, logos, and typography based on the verified tenant_id.
- **Custom Domain Routing:** Frontend load balancers route institutions to white-labeled instances via unique subdomains (e.g., scada.universityname.edu) while connecting to common infrastructure.
- **Modular Dashboards:** Facility managers can prioritize specific widgets, such as ESG reporting for sustainability-focused campuses or booking interfaces for utilization tracking.

6.4 SaaS Subscription & Billing Model

Commercial viability is ensured through a tiered billing engine that accounts for variable AI inference and messaging costs.

- **Automated Billing Integration:** Managed via Stripe to handle institution-wide recurring subscriptions and contract invoicing.
- **Usage-Based Metering:** The system tracks MQTT messaging volume and token budgets for Z.AI GLM API calls per tenant to manage variable operational expenses.
- **Tiered Licensing:**
 - **Standard Tier:** Provides core sensor ingestion, dashboard access, and baseline HVAC optimization.
 - **Premium Tier:** Unlocks advanced agentic capabilities, custom ESG policy generation, and predictive hardware maintenance with dedicated SLAs.

7.0 Enterprise Platform Migration Strategy

This section outlines the strategic roadmap to transition **SCADA-i** from a hackathon prototype into an enterprise-grade B2B SaaS platform. The engineering team will execute a phased migration from fragmented PaaS providers (Netlify, Railway, Supabase) to a unified enterprise cloud environment (AWS or GCP) to ensure high availability, strict compliance, and scalable multi-tenancy.

7.1 Frontend Edge Delivery Migration

The current frontend architecture, built with React and TypeScript, will migrate from Netlify to a globally distributed Content Delivery Network (CDN) such as **AWS CloudFront** (paired with S3) or **Google Cloud CDN**. This transition provides the advanced web application firewalls (WAF), DDoS protection, and edge-caching required to meet institutional Service Level Agreements (SLAs).

7.2 Containerization & Kubernetes Orchestration

To handle high-volume reasoning requests across multiple campuses, the Node.js, FastAPI, and LangGraph multi-agent architecture will migrate off Railway.

- **Strategy:** All backend services and specialized agents (HVAC, Booking, ESG) will be fully containerized using **Docker**.
- **Deployment:** These containers will be orchestrated via managed Kubernetes clusters (AWS EKS or GCP GKE), allowing the system to automatically spin up additional AI worker nodes in response to peak utilization.

7.3 Enterprise Data & Storage Transition

The centralized PostgreSQL database and document storage buckets will be migrated from Supabase to a dedicated, managed relational database service like **AWS RDS** or **Google Cloud SQL**.

- **Execution:** Data migration will utilize continuous replication tools (e.g., AWS Database Migration Service) to seamlessly sync the live state before cutting over, ensuring zero downtime.
- **Benefits:** This move unlocks Multi-AZ redundancy, automated point-in-time recovery, and deeper integration with IAM policies for strict multi-tenant **Row-Level Security (RLS)**.

7.4 Managed IoT Infrastructure Integration

As hardware installations scale, maintaining a standalone MQTT broker becomes a single point of failure. The messaging tier will migrate to a fully managed enterprise service like **AWS IoT**

Core. This allows SCADA-i to securely provision, authenticate, and ingest telemetry from deployed sensors with built-in, bidirectional hardware-to-cloud encryption.

7.5 Network Security & Private VPCs

Rather than relying on public internet routing between decoupled hosting providers, the entire enterprise **SCADA-i** ecosystem will be encapsulated within a Virtual Private Cloud (VPC). The databases, message brokers, and AI reasoning endpoints will communicate exclusively over private subnets, shielding the core infrastructure from public internet exposure.